Cross-Site Scripting (XSS)

Reflected XSS and Stored XSS — Week 6 VAPT Notes

Technical Learning Notes | Prepared from authorized local lab work and OWASP guidance

1. Definition and overview

Cross-Site Scripting (XSS) is a client-side web vulnerability in which attacker-controlled content is interpreted by a victim's browser as executable script in the security context of a trusted website. XSS commonly occurs when an application places untrusted data into a page without the correct context-sensitive output encoding or safe handling.XSS is different from SQL Injection: SQLi changes database query interpretation, while XSS causes the browser to interpret untrusted content as active page content or script. The effect depends on where the data is inserted, the browser context, the user's privileges, and the application's defenses.

2. Reflected XSS

Reflected XSS occurs when a request contains untrusted input and the server immediately includes that input in the response without appropriate encoding or safe handling. The malicious content is not necessarily stored by the application. A victim generally has to open a crafted link or submit a crafted request.

Illustrative example: a search page reads a query parameter and inserts it directly into an HTML response. If the application inserts input into a script-capable context without encoding, the browser may execute it. The safe response depends on context: HTML text, HTML attributes, JavaScript strings, URLs, and CSS require different handling. Prefer safe templating and context-aware encoding rather than manual string replacement.

3. Stored XSS

Stored XSS occurs when an application saves untrusted content—such as a guestbook message, comment, profile field, or support ticket—and later displays it to users in an executable context. The payload can affect multiple viewers who load the affected page, potentially including privileged users.

Storing content does not itself make it safe. The application must safely handle it every time it is rendered. A secure design encodes data for the output context, avoids unsafe DOM sinks, and uses a carefully designed sanitization policy if users are allowed to submit limited HTML.

4. Impact and risk

Possible impacts include actions performed in the context of a signed-in user, exposure of information accessible to the page, alteration of displayed content, phishing, and misuse of application functions. Session-token theft may be possible in some circumstances, but protections such as HttpOnly cookies can limit direct script access to cookies. XSS impact is context-dependent and should not be overstated without evidence.

5. Evidence from my authorized DVWA lab

In the local DVWA lab, the Reflected XSS exercise displayed a browser alert after the test input `<script>alert('Reflected XSS')</script>` was submitted. In the Stored XSS guestbook exercise, submitting `<script>alert('Stored XSS')</script>` caused a browser alert when the stored content was rendered. These observations demonstrate reflected and stored script execution in the intentionally vulnerable lab as configured.

Evidence to reference:
• `02-Screenshots/02-Reflected-XSS/` — browser alert evidence (use the actual saved filename).
• `02-Screenshots/03-Stored-XSS/` — guestbook alert evidence (use the actual saved filename).
Only refer to screenshots that are present in the final project folder. These results do not imply that other websites are vulnerable.

6. Prevention and secure design

1. Apply context-aware output encoding wherever untrusted data is rendered. Use a maintained framework's safe templating features.
2. Avoid dangerous DOM APIs and sinks such as assigning untrusted strings to `innerHTML`. Prefer `textContent` or safe DOM construction when HTML is not required.
3. If limited HTML is a business requirement, use a well-maintained HTML sanitizer with a strict allowlist and keep it updated.
4. Validate data according to business rules, but do not treat input validation as the primary XSS defense.
5. Deploy a restrictive Content Security Policy (CSP) as defense in depth. CSP does not replace output encoding.
6. Use appropriate cookie attributes, including HttpOnly, Secure, and SameSite where suitable. These reduce certain consequences but do not fix the XSS root cause.
7. Include reflected and stored XSS tests in development and security regression testing.

7. Verification and retesting

Test each input-to-output path separately. Confirm that script-like input is rendered as inert text or safely sanitized, not executed. For stored content, test both submission and later viewing, including relevant user roles. Review the actual rendering context and test normal content to ensure the fix has not broken intended behavior. Record the page, field, test case, observed behavior, and retest result.

8. Key takeaway

Reflected XSS returns unsafe input in the immediate response; Stored XSS saves unsafe content and later serves it to viewers. Context-aware output encoding and safe rendering are the primary defenses. In the local DVWA lab, the alert dialogs provided evidence of both behaviors.

References for further reading

- OWASP Cross Site Scripting Prevention Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- OWASP DOM based XSS Prevention Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/DOM_based_XSS_Prevention_Cheat_Sheet.html